

Asistente Editorial en VPS orquestado mediante n8n

Este proyecto consiste en un **Asistente Editorial automatizado desplegado en un VPS propio**, lo que permite que la solución permanezca disponible de forma continua, sin depender de que un computador local esté encendido.

La arquitectura combina **n8n** como motor de orquestación, **Telegram** como interfaz principal con el usuario, **Airtable** como memoria estructurada y repositorio operativo, **Gemini** como motor de IA, **Google Docs/Drive** como destino de los borradores y **Gmail** como canal de notificación cuando efectivamente se crea un borrador.

El sistema se ejecuta detrás de un dominio propio con **HTTPS**, y las credenciales de las distintas APIs se gestionan dentro de n8n, separadas de la lógica del workflow.

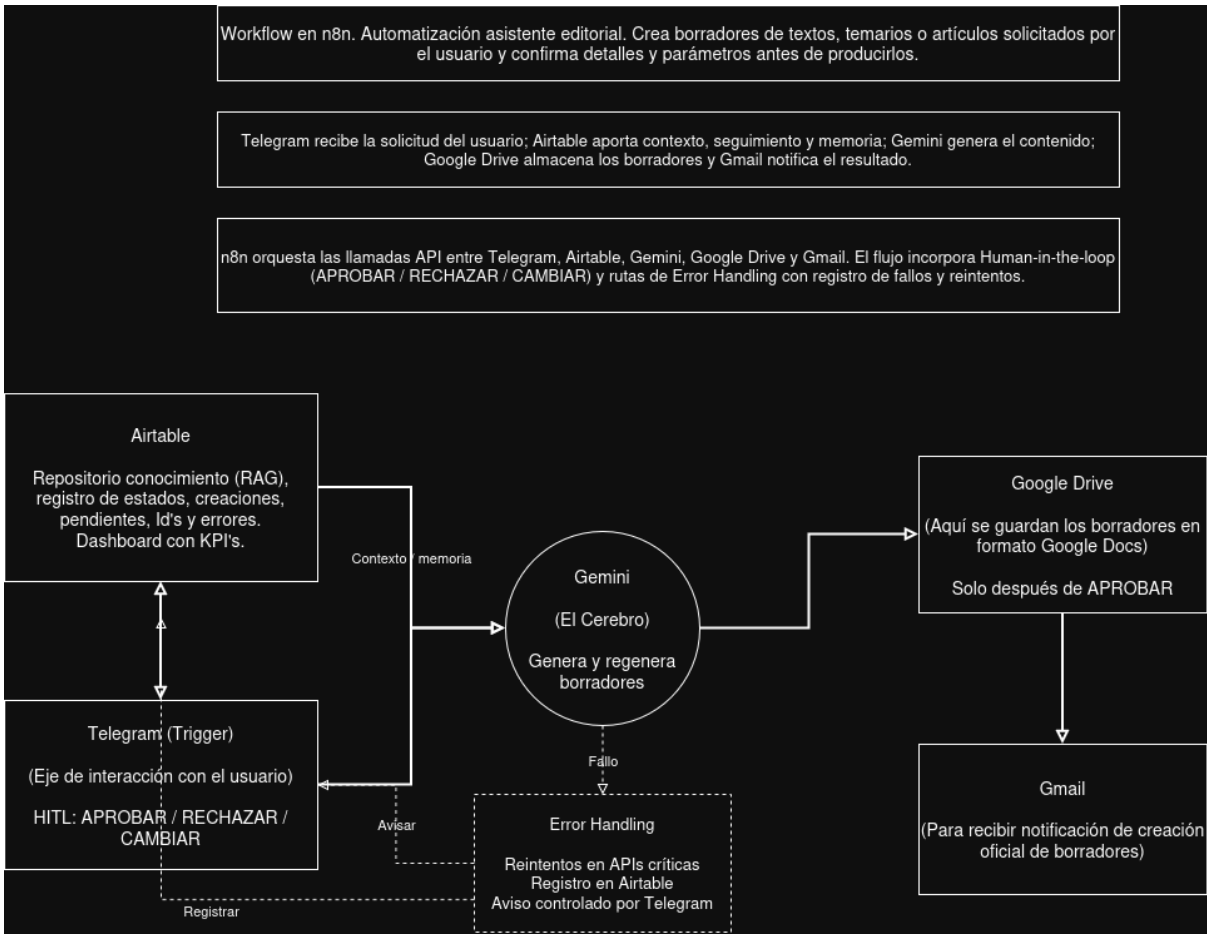
En funcionamiento, el usuario envía una solicitud por Telegram y n8n coordina el proceso completo: recupera contexto y lineamientos desde Airtable, recopila parámetros como tipo de texto, tema, extensión y categoría, construye un prompt dinámico y llama a Gemini para generar el borrador.

Dentro de n8n, un router interpreta y deriva cada solicitud según su intención —CREAR, GUARDAR, CONSULTAR, ANALIZAR o ESTADO— para ejecutar únicamente la ruta correspondiente.

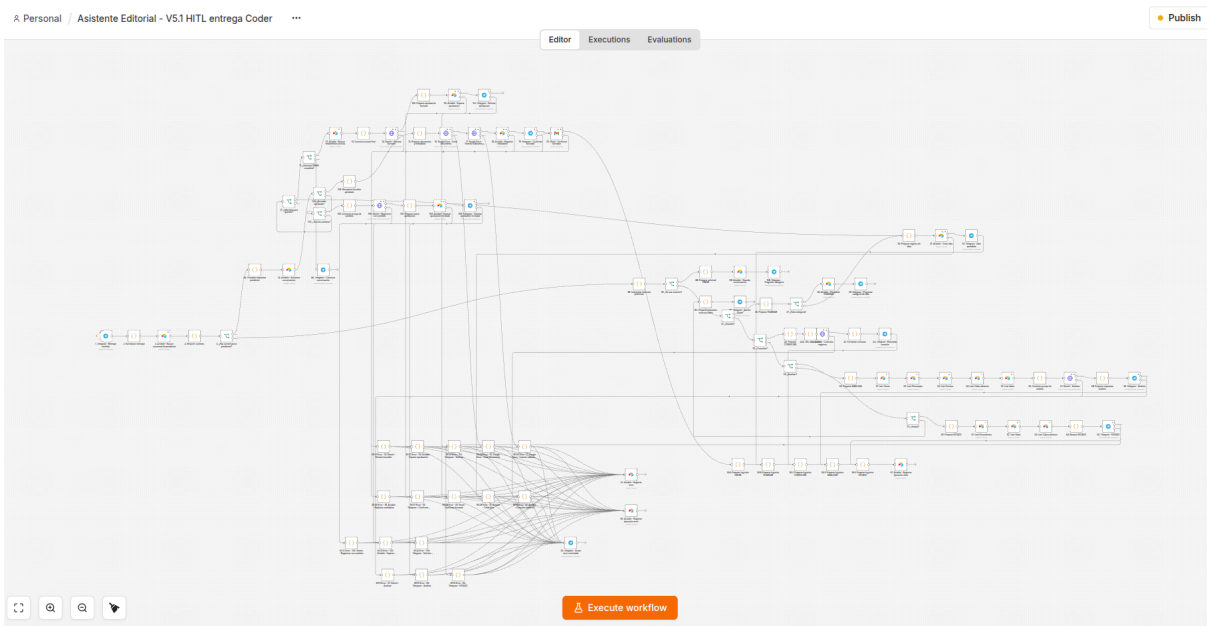
Antes de crear el documento definitivo, el flujo incorpora un **Human-in-the-loop** que permite **APROBAR, RECHAZAR o CAMBIAR** el contenido. Solo tras la aprobación se crea el documento en Google Docs, se registran sus metadatos en Airtable y se envían confirmaciones por Telegram y Gmail.

La solución incluye además **reintentos, rutas de Error Handling, registro de incidentes y métricas de ejecución**, de modo que no es solo un workflow de generación de texto, sino un pequeño ecosistema de automatización autónomo, persistente y trazable.

Arquitectura Simplificada de la Solución:



Arquitectura Completa: La imagen inferior corresponde al workflow completo implementado en n8n. El diagrama superior representa su arquitectura funcional simplificada.



Manual operativo de datos

1. Objetivo de la base de datos

Airtable funciona como la **memoria estructurada y repositorio operativo** del Asistente Editorial. No almacena el contenido completo de los documentos finales —estos viven en Google Docs—, sino los metadatos, reglas, estados conversacionales, entidades narrativas, errores y métricas necesarias para que n8n pueda recuperar contexto y tomar decisiones durante la ejecución.

La base también permite mantener trazabilidad: qué se creó, con qué parámetros, qué quedó pendiente, qué aprobó el usuario, qué errores ocurrieron y qué relaciones existen entre documentos, ideas y entidades narrativas.

2. Tablas principales

Tabla	Función
Ideas	Almacena ideas capturadas desde Telegram u otras fuentes. Incluye título, contenido, categoría, proyecto, estado, fuente y fecha de creación. Una idea puede vincularse posteriormente a uno o más documentos.
Documentos	Registra los metadatos de los borradores creados: título, tipo, categoría, tema, extensión solicitada/final, estado, resumen, lineamientos, URL de Google Docs y fecha de creación. El texto completo permanece en Google Docs.
Lineamientos	Contiene reglas permanentes que n8n recupera antes de generar un borrador. Permite definir ámbito, prioridad, estado activo y contenido del lineamiento.
Taxonomía	Catálogo de categorías, subcategorías y proyectos. Su objetivo es mantener clasificación consistente del contenido.
Canon	Registra hechos canónicos aprobados explícitamente por el autor. Permite separar información definitiva de ideas o propuestas todavía no aceptadas.
Personajes	Base de referencia para personajes narrativos, incluyendo rol, descripción, datos canónicos, estado y notas.
Lugares	Almacena lugares relevantes para proyectos narrativos y sus datos descriptivos o canónicos.
Eventos	Registra acontecimientos, cronología, consecuencias y estado canónico dentro de los proyectos narrativos.

Funcionamiento de Conversaciones pendientes

Esta tabla es clave para el comportamiento conversacional del asistente.

Cuando el usuario realiza una solicitud incompleta, n8n guarda temporalmente:

- Chat ID
- Intención
- Paso pendiente
- Payload JSON
- Activa
- Última actualización

Ejemplo simplificado de estado pendiente:

```
{
  "accion": "CREAR",
  "tipo": "Artículo",
  "tema": "disciplina personal",
  "extension": 800,
  "categoria": "Escritos",
  "lineamientos_especificos": "tono directo",
  "draft_title": "Disciplina y ejecución",
  "aprobacion_humana": false
}
```

Gracias a este mecanismo, el usuario puede responder posteriormente con **APROBAR**, **RECHAZAR**, **CAMBIAR** o completar datos que todavía falten sin reiniciar la conversación.

Esquema de transferencia hacia Gemini

Antes de invocar al modelo, n8n construye dinámicamente el prompt utilizando variables del sistema.

Conceptualmente, los datos de entrada tienen esta estructura:

```
{
  "tipo": "Artículo",
  "tema": "disciplina personal",
  "extension": 800,
  "categoria": "Escritos",
  "subcategoria": "",
  "proyecto": "",
  "lineamientos_especificos": "tono directo y austero",
  "lineamientos_permanentes": [
```

```

    "No inventar fuentes",
    "Mantener la extensión solicitada"
  ]
}

```

n8n transforma esos datos en el cuerpo JSON requerido por la API de Gemini.

Ejemplo simplificado:

```

{
  "contents": [
    {
      "parts": [
        {
          "text": "PROMPT DINÁMICO CON LAS VARIABLES DEL SISTEMA"
        }
      ]
    }
  ],
  "generationConfig": {
    "temperature": 0.6,
    "maxOutputTokens": 8192
  }
}

```

Esquema de transferencia hacia Google Docs

Una vez aprobado el borrador por el usuario, n8n prepara los datos necesarios para crear el documento en Google Docs. Ejemplo simplificado:

```

{
  "title": "Título del borrador",
  "documentId": "ID generado por Google Docs",
  "status": "BORRADOR"
}

```

Flujo de datos de salida

Cuando Gemini devuelve el borrador:

1. n8n extrae título y cuerpo.
2. calcula la extensión final;
3. guarda temporalmente el borrador en **Conversaciones pendientes**;
4. lo presenta al usuario mediante Telegram;
5. el usuario decide **APROBAR**, **RECHAZAR** o **CAMBIAR**;
6. solo después de **APROBAR** se crea el documento en Google Docs;
7. Airtable registra sus metadatos;
8. Telegram y Gmail notifican la creación.

Así se evita crear documentos definitivos sin revisión humana.

Registro de errores y ejecuciones

La tabla **Errores** centraliza fallos técnicos y permite analizar:

- nodo afectado;
- ruta;
- mensaje;
- tipo de error;
- si hubo reintento;
- estado de resolución.

La tabla **Ejecuciones** sirve como base para el dashboard de control:

- cantidad total de ejecuciones;
- éxitos;
- errores;
- tasa de error;
- rutas utilizadas;
- aprobaciones humanas.

Cuadro comparativo y matriz de costos

Conviene dejar claro que **no seleccioné el modelo más barato**, sino el modelo que ofrece un equilibrio razonable entre costo, velocidad, calidad y simplicidad de integración.

El workflow actual utiliza **Gemini 3.7 Flash** para generación y regeneración de borradores. Google lo presenta como un modelo Flash de alta velocidad orientado a uso de herramientas y ejecución confiable de múltiples pasos. A septiembre de 2026, su precio estándar de pago es de **USD 0,75 por millón de tokens de entrada y USD 3,75 por millón de tokens de salida**, con nivel gratuito disponible

Matriz comparativa

Modelo	Entrada / 1M tokens	Salida / 1M tokens	Uso posible en el proyecto	Evaluación para este caso
Gemini 3.7 Flash	USD 0,75	USD 3,75	Generación de borradores, revisión y regeneración	Elegido. Buen equilibrio entre velocidad, calidad, capacidad multi-step y costo. Además ya está integrado y probado en n8n.
GPT-4o mini	USD 0,15	USD 0,60	Clasificación, extracción, etiquetas, tareas simples	Mucho más barato para tareas pequeñas y estructuradas, pero el proyecto no necesita añadir otro proveedor para esas operaciones. OpenAI lo posiciona como modelo

				rápido y económico para tareas enfocadas.
Claude Sonnet 5	USD 2,00	USD 10,00	Escritura extensa, revisión compleja, razonamiento editorial profundo	Alta capacidad, pero su costo por token es significativamente mayor que Gemini Flash para este caso. Anthropic publica además ahorro de hasta 50% mediante batch.
Claude Sonnet 4.6	USD 3,00	USD 15,00	Lectura densa y razonamiento complejo	Muy capaz, pero sobredimensionado y más costoso para la generación editorial cotidiana del workflow.

Decisión adoptada

Se seleccionó **Gemini 3.7 Flash** como motor principal porque:

- ofrece velocidad adecuada para una interacción conversacional por Telegram;
- soporta generación de textos extensos;
- funciona correctamente con prompts dinámicos y múltiples variables;
- tiene un costo considerablemente menor que modelos Sonnet de mayor capacidad;
- ya estaba disponible dentro del ecosistema utilizado en el proyecto;
- evita sumar complejidad operacional mediante múltiples proveedores y credenciales;
- ha sido probado satisfactoriamente en generación, regeneración y Human-in-the-loop.

La elección no impide utilizar otros modelos en futuras versiones. Por ejemplo, GPT-4o mini sería una alternativa eficiente para clasificación masiva o extracción estructurada, mientras que Claude podría reservarse para procesos puntuales de lectura o análisis especialmente complejos.

Estrategia de optimización de costos

El sistema incorpora varias decisiones concretas para limitar consumo:

- **Gemini solo se invoca cuando es necesario generar o regenerar contenido.**
- Acciones como **APROBAR**, **RECHAZAR**, consultas de estado o lectura de registros se resuelven mediante lógica de n8n y Airtable, sin llamar al modelo.
- La opción **CAMBIAR** sí genera una nueva llamada a Gemini, por lo que Telegram advierte explícitamente al usuario que solicitar cambios implica mayor consumo de tokens.
- El prompt utiliza solo los lineamientos aplicables al contexto, evitando enviar información innecesaria.
- La extensión solicitada limita la longitud de la salida.
- Google Docs, Gmail y Telegram se ejecutan mediante sus APIs sin intervención del LLM.

- En caso de procesos masivos futuros, se podría evaluar Batch API. Tanto Google como Anthropic ofrecen modalidades batch con precios inferiores a ejecución estándar en determinados modelos.

Ejemplo sencillo de costo

Suponiendo una generación con aproximadamente:

- 2.000 tokens de entrada;
- 2.000 tokens de salida;

con Gemini 3.7 Flash el costo aproximado sería:

Entrada:

$2.000 / 1.000.000 \times \text{USD } 0,75 \approx \text{USD } 0,0015$

Salida:

$2.000 / 1.000.000 \times \text{USD } 3,75 \approx \text{USD } 0,0075$

Total aproximado: USD 0,009 por generación.

Por lo tanto, incluso 100 generaciones de ese tamaño rondarían aproximadamente **USD 0,90**, antes de considerar cualquier nivel gratuito disponible.

Si el usuario solicita **CAMBIAR**, cada regeneración agrega aproximadamente otro ciclo de consumo comparable. Por eso el sistema avisa al usuario antes de utilizar esa opción.

Fuentes de referencia de precios (consultadas en septiembre de 2026):

Google Gemini API Pricing: <https://ai.google.dev/gemini-api/docs/pricing>

OpenAI API Pricing: <https://openai.com/api/pricing/>

Anthropic Claude Pricing: <https://docs.anthropic.com/en/docs/about-claude/pricing>

Documentación de seguridad y resiliencia

Minimización de datos

- Telegram envía solo lo necesario para procesar la solicitud.
- Airtable guarda metadatos, estados y referencias, no el contenido completo de todos los documentos.
- El contenido largo queda en Google Docs.
- Las credenciales no se guardan en el JSON del workflow.

Seguridad

- n8n corre sobre HTTPS.
- Gmail, Google Docs, Airtable, Telegram y Gemini usan credenciales gestionadas por n8n.
- OAuth/API keys quedan separadas de la lógica del workflow.
- El JSON exportado conserva referencias a credenciales, no las claves reales.

Resiliencia

- Reintentos en APIs críticas.
- Rutas de error controladas.
- Registro de fallos en la tabla **Errores**.
- Aviso al usuario por Telegram si ocurre un fallo controlado.
- Tabla **Ejecuciones** para trazabilidad y métricas.

Human-in-the-loop

- **APROBAR** → permite crear el Google Doc.
- **RECHAZAR** → cancela.
- **CAMBIAR:** → regenera el borrador con nuevas instrucciones.
- El sistema no crea el documento oficial hasta aprobación humana.
- Se avisa que pedir cambios genera consumo adicional de tokens.

Protecciones adicionales

- Validación de extensión.
- Estados conversacionales.
- Comparación correcta de tipos.
- Flujo controlado para evitar pasos ambiguos o bucles conversacionales.

El dashboard público de Airtable permite monitorear las principales métricas operativas del sistema: total de ejecuciones, tasa de error, aprobaciones humanas, distribución de ejecuciones exitosas/fallidas, ejecuciones por ruta, errores por tipo y estado, y documentos registrados.

LINK A DASHBOARD DE AIRTABLE

<https://airtable.com/appbrFkc7zcVAgB0r/shr0Bizm6T4UAp8qY>

Checklist técnico

- **Prevención de bucles infinitos:** las conversaciones usan estados controlados y pasos pendientes definidos; las regeneraciones requieren una nueva instrucción humana explícita.
- **Validación de tipos:** los filtros comparan booleanos, strings y números según el tipo de dato esperado.
- **Prompt dinámico:** el prompt se construye en n8n utilizando variables como tipo, tema, extensión, categoría, proyecto y lineamientos. Esto ya está descrito en tu manual.